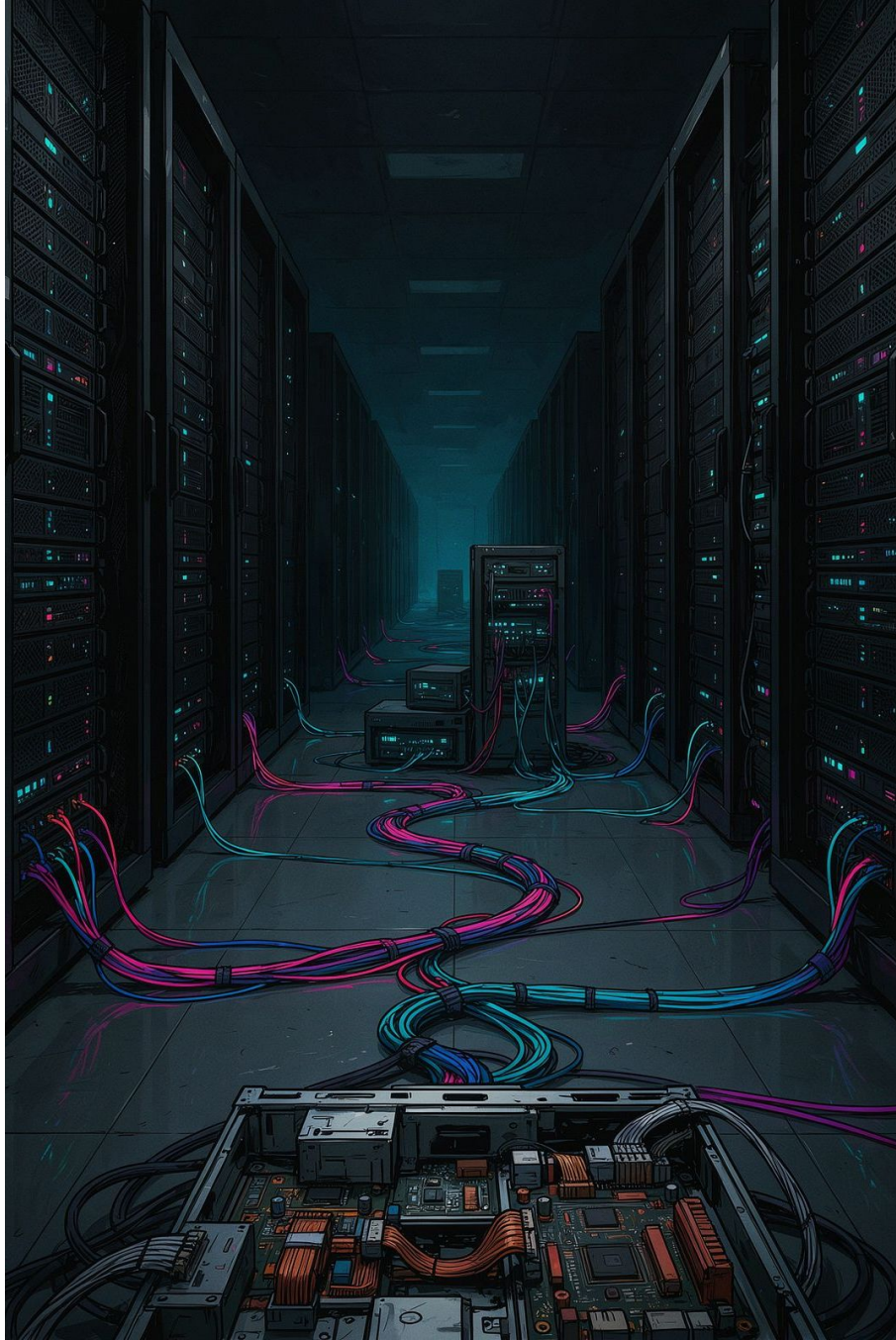
Веб-технологии, безопасность и Docker



Содержание лекции

Что сегодня разберём

01

Как работает сайт

Клиент-серверная модель, HTTP, DNS и путь запроса от браузера до базы данных.

02

Серверная архитектура

Reverse proxy, балансировка нагрузки, масштабирование компонентов.

03

Безопасность веб-приложений

Уязвимости, модель доверия, MITRE ATT&CK и защитные меры.

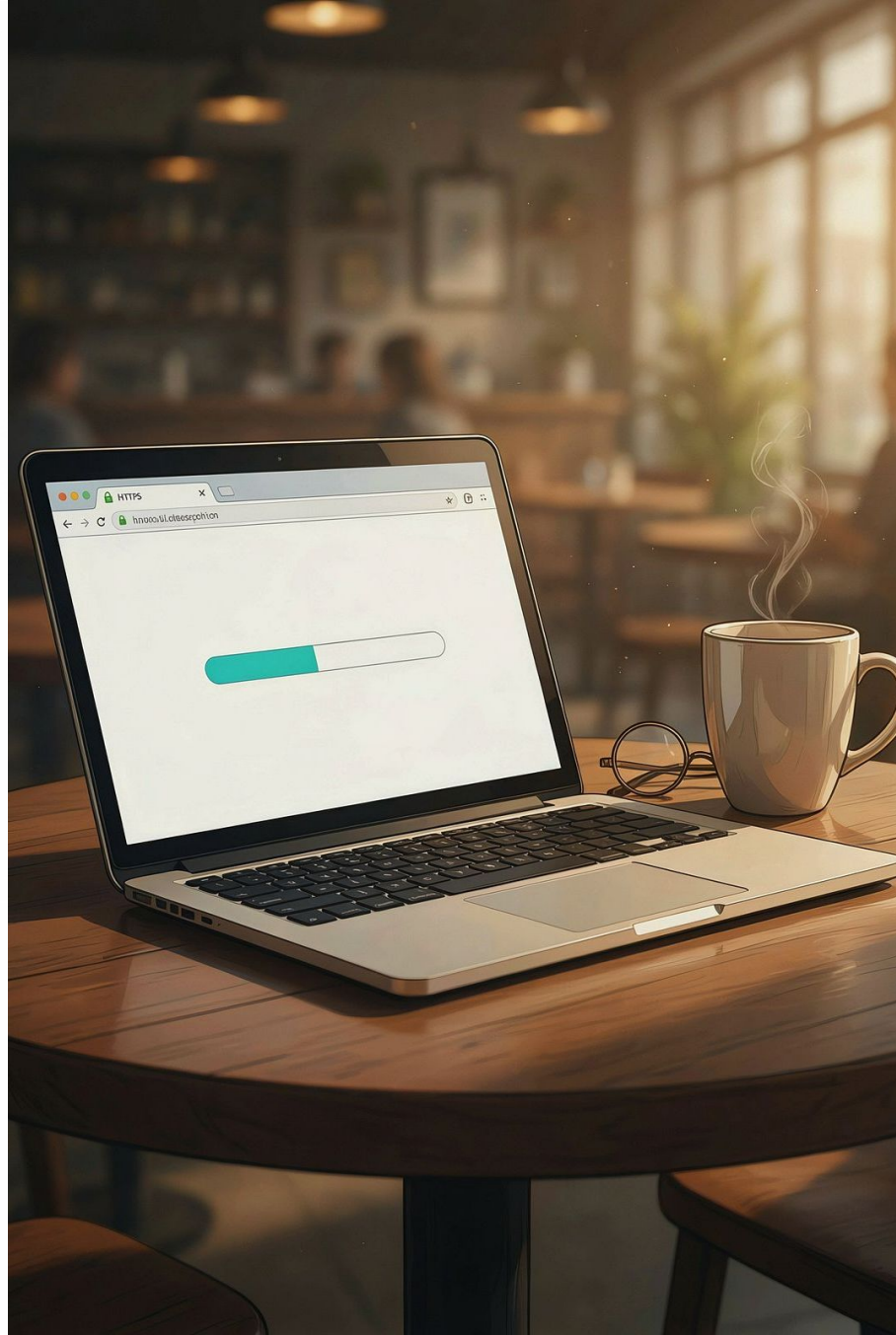
04

Docker и контейнеры

Образы, контейнеры, Dockerfile и docker compose для учебного стенда.

Как работает сайт

Сайт — это **клиент-серверное приложение**. Браузер пользователя выступает клиентом: он запрашивает данные, получает ответ и отображает интерфейс. Сервер принимает запрос, выполняет бизнес-логику, при необходимости обращается к базе данных и возвращает ответ. Каждое взаимодействие проходит через несколько уровней инфраструктуры, прежде чем пользователь увидит результат.



Путь запроса от браузера до базы данных



Каждый шаг этого пути имеет значение для производительности и безопасности. Понимание полной цепочки помогает находить узкие места и правильно расставлять защитные меры.

Пример: https://example.org/catalog

1 DNS-разрешение

DNS сопоставляет доменное имя с IP-адресом сервера.

2 Установка соединения

Браузер подключается к серверу; при HTTPS сначала согласовывается защищённый TLS-канал.

3 HTTP-запрос

Браузер отправляет `GET /catalog` — веб-сервер принимает и передаёт приложению.

4 Обработка и ответ

Приложение проверяет права, получает данные из БД и возвращает HTML, CSS, JS или JSON.

HTTP в двух словах

HTTP — протокол обмена сообщениями между клиентом и сервером. Каждый запрос содержит **метод**, путь, заголовки, cookies и, при необходимости, тело. Метод определяет намерение запроса.

Метод	Назначение
`GET`	Получить данные без изменения ресурса
`POST`	Создать ресурс или выполнить действие
`PUT / PATCH`	Изменить ресурс полностью или частично
`DELETE`	Удалить ресурс

HTTP-статусы: что отвечает сервер

200 OK

Запрос успешно выполнен, данные возвращены.

401 Unauthorized

Требуется аутентификация — пользователь не вошёл в систему.

403 Forbidden

Доступ запрещён — пользователь идентифицирован, но прав нет.

404 Not Found

Ресурс не найден по указанному пути.

500 Server Error

Внутренняя ошибка приложения на сервере.

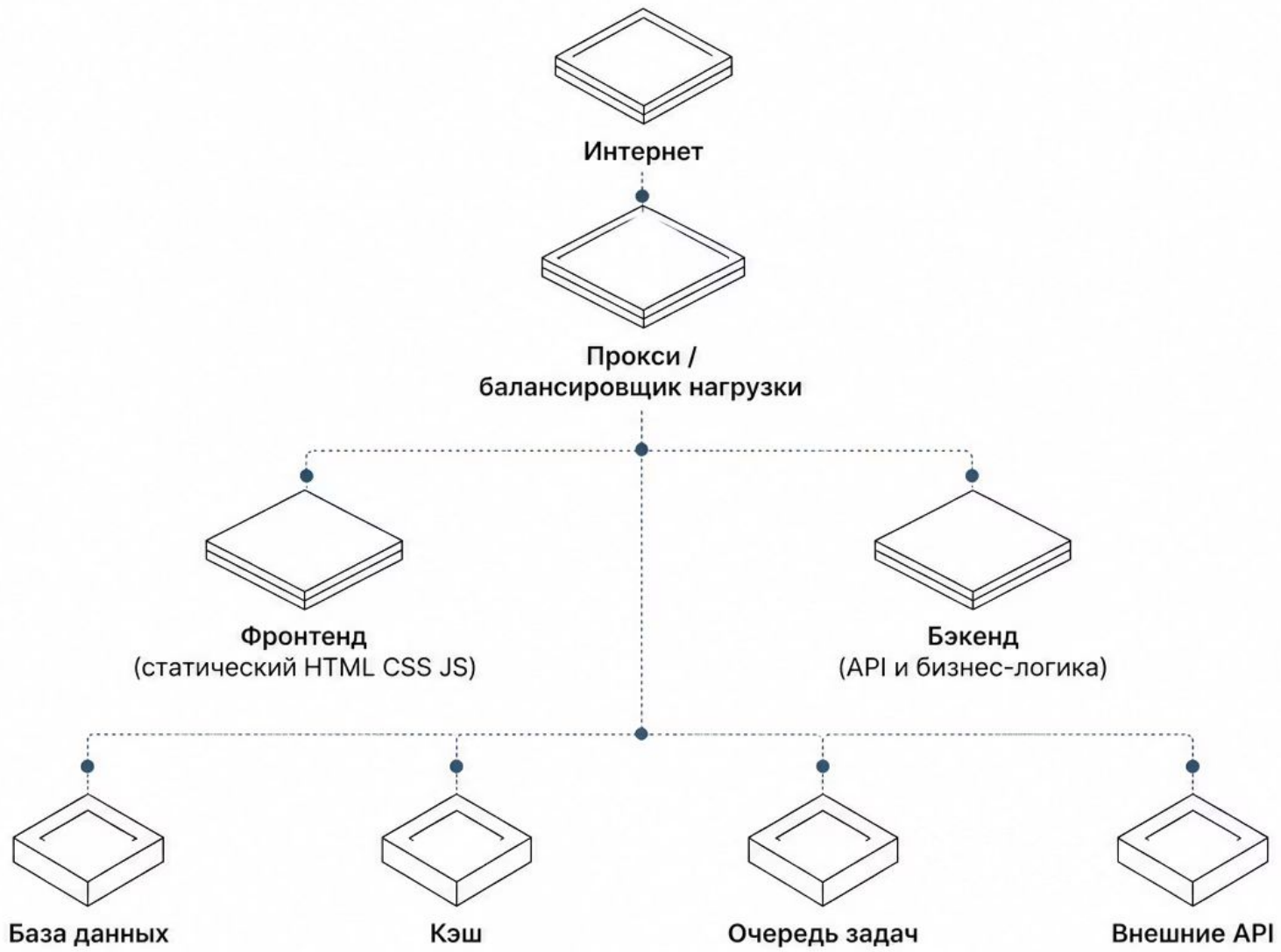
HTTP сам по себе не хранит состояние. Приложения используют cookies, серверные сессии или токены, чтобы «помнить» пользователя между запросами.



РАЗДЕЛ 2

Серверная архитектура

Компоненты типичной архитектуры



Для небольшого учебного приложения все части могут работать на одном сервере. В реальном проекте компоненты разделяют, чтобы повысить надёжность, производительность и безопасность.

Proxy: зачем он нужен

Что такое Proxy

Proxy, например **Nginx**, принимает внешние запросы и перенаправляет их внутренним сервисам. Для клиента он выглядит как единая точка входа.

Что он умеет

- Завершать TLS и обслуживать HTTPS
- Раздавать статические файлы напрямую
- Ограничивать частоту запросов (rate limiting)
- Вести централизованные логи
- Скрывать внутренние сервисы от прямого доступа из интернета

Масштабирование: когда одного сервера мало



Горизонтальное масштабирование

Запускают несколько одинаковых экземпляров бэкенда, а балансировщик нагрузки распределяет входящие запросы между ними.



Масштабирование базы данных

БД масштабируют осторожнее: применяют резервное копирование, репликацию, кэширование и разделение чтения и записи.



Кэширование

Кэш (например Redis) снижает нагрузку на БД, сохраняя результаты часто запрашиваемых данных в памяти.

РАЗДЕЛ 3

Фронтенд, бэкенд и база данных



Фронтенд: что запускается в браузере

HTML

Описывает структуру и содержимое страницы — заголовки, параграфы, формы, ссылки.

CSS

Определяет внешний вид: цвета, шрифты, отступы, расположение элементов.

JavaScript

Добавляет интерактивность: реагирует на действия пользователя и отправляет запросы к API.

⚠ Фронтенд — недоверенная зона. Пользователь может изменить код в браузере, параметры запроса и локальное хранилище. Проверки доступа и работа с секретами должны выполняться **только на сервере**.

Бэкенд: серверная логика

Что делает бэкенд

Бэкенд — серверное приложение на Python, JavaScript/Node.js, Java, Go, PHP и других языках. Он реализует бизнес-логику: регистрирует пользователей, проверяет права, обрабатывает заказы и формирует API-ответы.

API-ответы часто используют **формат JSON**:

```
{ "id": 42, "name": "Учебный CTF", "published": true }
```

Бэкенд отвечает за

- Аутентификацию и авторизацию
- Валидацию входных данных
- Взаимодействие с БД
- Хранение секретов
- Интеграцию с внешними сервисами

База данных: хранение данных приложения



Реляционные БД (SQL)

PostgreSQL и MySQL хранят данные в таблицах со строгой схемой. Используют SQL для запросов. Подходят для структурированных данных с отношениями.



Документные БД (NoSQL)

MongoDB хранит данные как JSON-подобные документы. Гибкая схема, удобна для неоднородных данных и быстрого прототипирования.

Обращайтесь к БД через **параметризованные запросы** или безопасный ORM. Никогда не помещайте строки подключения и пароли в исходный код или публичный репозиторий.

РАЗДЕЛ 4

Базовая модель безопасности веб-приложения



Три границы доверия

1. Пользовательский ввод

Любые параметры URL, данные форм, заголовки, cookies и загружаемые файлы считаются **недоверенными**. Каждый из них необходимо валидировать и экранировать перед использованием.

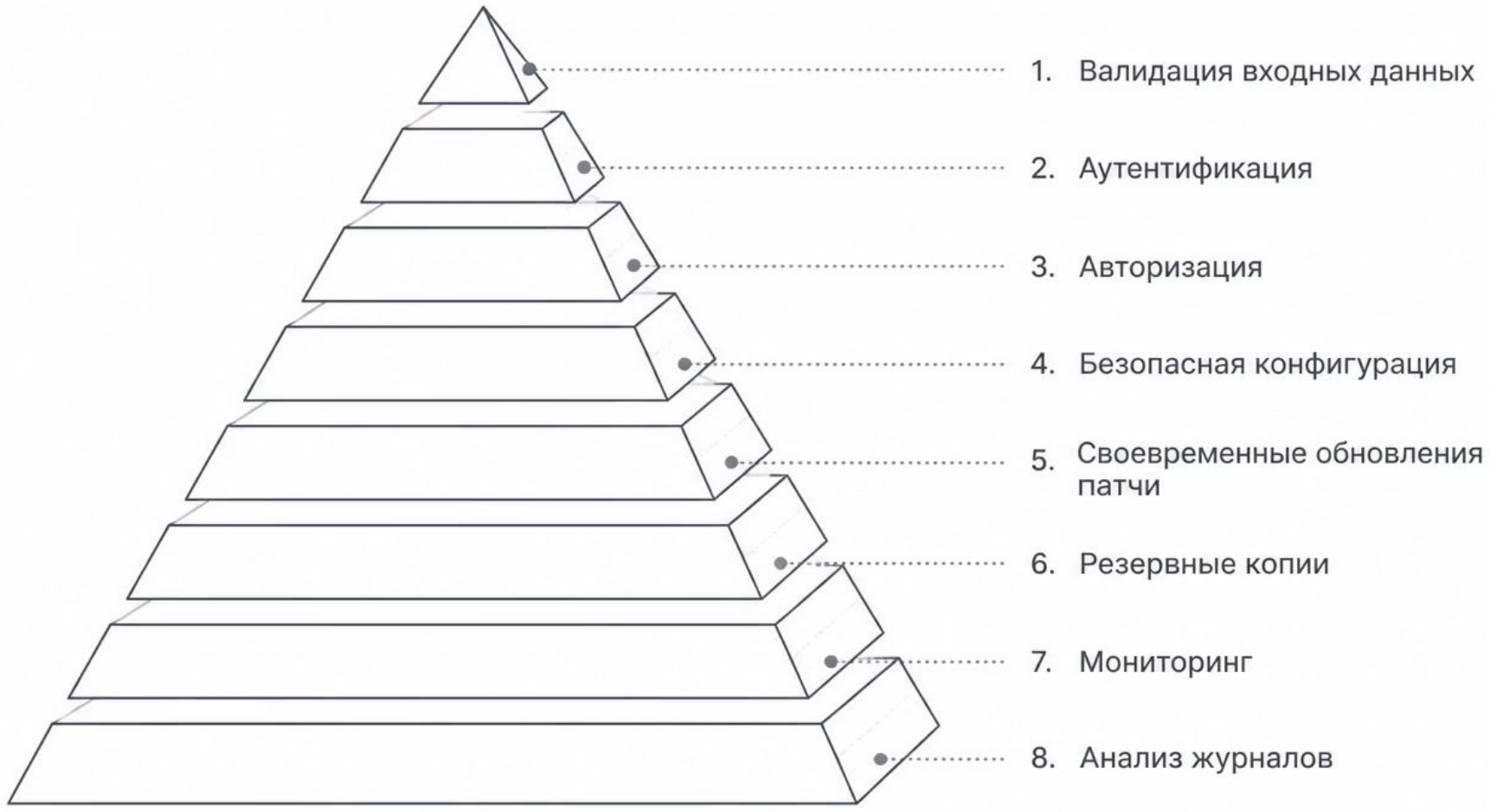
2. Связь между сервисами

Внутренний сервис тоже должен проверять, **кто** к нему обращается и **какие данные** получает. Не все внутренние запросы безопасны по умолчанию.

3. Данные и секреты

Пароли, токены, ключи и персональные данные требуют отдельного контроля доступа, журналирования и **минимальных привилегий**.

Защита строится слоями



Ни один слой защиты не является абсолютным. Именно поэтому применяют принцип **«глубокой защиты»**: компрометация одного уровня не должна приводить к полному взлому системы.

РАЗДЕЛ 5

Основные веб-уязвимости

Ниже перечислены распространённые классы проблем. Их следует изучать и демонстрировать **только** на локальном намеренно уязвимом приложении.



Аутентификация vs. Авторизация

Аутентификация

«Кто ты?» — процесс проверки личности пользователя. Логин и пароль, токен, сертификат или биометрия. Результат: система знает, кто выполняет запрос.

Авторизация

«Что тебе разрешено?» — проверка прав конкретного пользователя на конкретное действие или объект. Даже корректный вход не даёт права читать чужие данные.

Ошибка разработчиков: проверить аутентификацию и забыть про авторизацию. Оба шага обязательны и выполняются на сервере.

Broken Access Control / IDOR

Суть проблемы

Пользователь получает доступ к чужому ресурсу из-за **отсутствия серверной проверки прав**. Например, изменив `?id=42` на `?id=43`, он читает данные другого пользователя.

Базовая защита

- Проверять право доступа к каждому объекту на сервере
- Применять принцип минимальных привилегий
- Не полагаться на скрытость URL как на защиту
- Использовать непредсказуемые идентификаторы (UUID)

SQL-инъекция

Суть проблемы

Пользовательские данные попадают напрямую в SQL-команду и **изменяют её логику**. Атакующий может читать, изменять или удалять данные в БД.

```
SELECT * FROM users WHERE name = ''+OR+1=1--'
```

Базовая защита

- Использовать параметризованные запросы
- Применять безопасный ORM
- Создавать отдельного пользователя БД с минимальными правами
- Никогда не конкатенировать пользовательский ввод в SQL-строку

XSS – межсайтовый скриптинг

Суть проблемы

Недоверенные данные отображаются в браузере как **активный JavaScript-код**. Атакующий похищает сессии, перенаправляет пользователей или изменяет содержимое страницы.

Базовая защита

- Контекстное экранирование вывода (HTML, JS, URL)
- Заголовок Content Security Policy (CSP)
- Безопасная работа с DOM: не использовать `innerHTML`
- Валидация на сервере перед сохранением данных

Ошибки аутентификации и сессий

Слабые пароли

Пользователи и системы используют простые или повторяющиеся пароли без ограничений.

Неправильная проверка токенов

Подпись JWT не проверяется, алгоритм «none» принимается сервером.

Предсказуемые сессии

Session ID генерируется на основе времени или последовательного счётчика.

Защита

MFA, безопасное хранение хешей паролей, ограниченный срок жизни токенов, серверная проверка подписи.

SSRF, небезопасная загрузка файлов и ошибочная конфигурация

Небезопасная загрузка файлов

Загруженный файл сохраняется или исполняется небезопасно. Защита: белый список форматов, проверка содержимого, хранение вне web-root, случайные имена.

SSRF

Сервер делает запросы по адресу, контролируемому пользователем. Защита: белый список адресов, сегментация сети, блокировка служебных диапазонов.

Ошибочная конфигурация

Открытые панели администрирования, тестовые режимы, подробные ошибки, лишние порты. Защита: инвентаризация, безопасные значения по умолчанию, закрытие неиспользуемых интерфейсов.



РАЗДЕЛ 6

MITRE ATT&CK и веб-безопасность

Что такое MITRE ATT&CK

Определение

MITRE ATT&CK — база знаний о тактиках и техниках противника. Это не чек-лист уязвимостей и не набор эксплойтов, а **язык для описания** наблюдаемого поведения атакующих.

Ключевые понятия

- **Тактика** — цель действия («зачем»): первоначальный доступ, сбор данных, эксфильтрация.
- **Техника** — способ достижения цели («как»): эксплуатация публичного приложения, использование корректных учётных записей.
- Помогает формулировать вопросы: «Что мы видим?», «Какие логи у нас есть?», «Что снизит риск?»

ATT&CK в контексте веб-приложений

Наблюдение	Контекст ATT&CK	Действие защитника
Необычная активность на публичном сервисе	Initial Access — T1190 Exploit Public-Facing Application	Исправить уязвимость, ограничить доступ, сохранить логи
Вход с корректной учётной записью в необычное время	Valid Accounts	Проверить сессии, MFA, историю входов и права
Массовое перечисление объектов или API-методов	Discovery	Включить логи, rate limiting, серверную авторизацию
Необычная выгрузка данных	Collection / Exfiltration	Проверить права, объём и направление передачи; ограничить экспорт

ATT&CK помогает команде перейти от реакции «что-то сломалось» к проактивному вопросу: «Какую технику противника мы сможем заметить в наших логах?»

Как читать запись CVE: пример Log4Shell

CVE-2021-44228
Apache Log4j2: удалённое выполнение кода через JNDI lookup
CVSS v3.1: 10.0 (Critical)
CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H
Затронуто: Log4j 2.0-beta9 – 2.14.1 | Исправлено: 2.15.0

CVE-2021-44228

Идентификатор: CVE-ГОД-НОМЕР. Единый ключ для поиска в NVD, вендорских бюллетенях и сканерах.

Описание

Компонент Log4j2, JNDI lookup в логируемых строках, результат — удалённое выполнение кода (RCE).

Затронутые версии

Проверьте, какая версия стоит у вас, и сверьте её с версией исправления 2.15.0.

Как читать CVSS: оценка и вектор

CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H

CVSS 10.0 Critical

Оценка серьёзности от 0 до 10. Это не вероятность атаки, а тяжесть последствий.

Вектор: AV:N / AC:L / PR:N / UI:N

Атака по сети, низкая сложность, без привилегий и без действий пользователя.

Вектор: S:C / C:H / I:H / A:H

Выход за границы компонента; высокое влияние на конфиденциальность, целостность и доступность.

Читайте CVE в порядке: что уязвимо → какие версии → как атаковать (вектор) → чем это грозит → что патчить.

National Vulnerability Database

Base Score Metrics

Exploitability Metrics

Attack Vector (AV)*

Network (AV:N)	Adjacent Network (AV:A)	Local (AV:L)	Physical (AV:P)

Attack Complexity (AC)*

Low (AC:L)	High (AC:H)

Privileges Required (PR)*

None (PR:N)	Low (PR:L)	High (PR:H)

User Interaction (UI)*

None (UI:N)	Required (UI:R)

Scope (S)*

Unchanged (S:U)	Changed (S:C)

Impact Metrics

Confidentiality Impact (C)*

None (C:N)	Low (C:L)	High (C:H)

Integrity Impact (I)*

None (I:N)	Low (I:L)	High (I:H)

Availability Impact (A)*

None (A:N)	Low (A:L)	High (A:H)

* - All base metrics are required to generate a base score.



РАЗДЕЛ 7

Docker: зачем нужны контейнеры

Что такое Docker

Docker **упаковывает приложение и его зависимости** в образ. Из образа запускается контейнер — изолированный процесс с собственными файловой системой, сетью и настройками. Команда и сервер запускают одну и ту же версию приложения, что устраняет проблему «у меня работает».

1

Образ (Image)

Неизменяемый шаблон приложения с кодом и зависимостями

2

Контейнер

Запущенный экземпляр образа — изолированный процесс

3

Registry

Хранилище образов: Docker Hub или внутренний реестр

Контейнер vs. Виртуальная машина

Характеристика	Контейнер Docker	Виртуальная машина
Ядро ОС	Использует ядро хостовой ОС	Запускает гостевую ОС с собственным ядром
Скорость запуска	Секунды или доли секунды	Десятки секунд и более
Потребление ресурсов	Минимальный оверхед	Резервирует CPU и RAM для гостевой ОС
Изоляция	Зависит от настройки хоста и контейнера	Сильнее за счёт гипервизора
Лучший сценарий	Отдельные сервисы, микросервисы	Полная изоляция ОС, несовместимые ядра

⚠ Контейнер — не «мини-ВМ» и не автоматическая защита. Не запускайте контейнеры от root без необходимости и не монтируйте Docker socket в приложение.

Основные сущности Docker



Image

Неизменяемый шаблон приложения. Содержит код, runtime, зависимости и конфигурацию. Собирается из Dockerfile.



Container

Запущенный экземпляр образа. Изолированный процесс с собственной файловой системой и сетью.



Volume

Постоянное хранилище данных вне жизненного цикла контейнера. Данные сохраняются при перезапуске.



Network

Изолированная виртуальная сеть для взаимодействия контейнеров между собой без выхода в интернет.

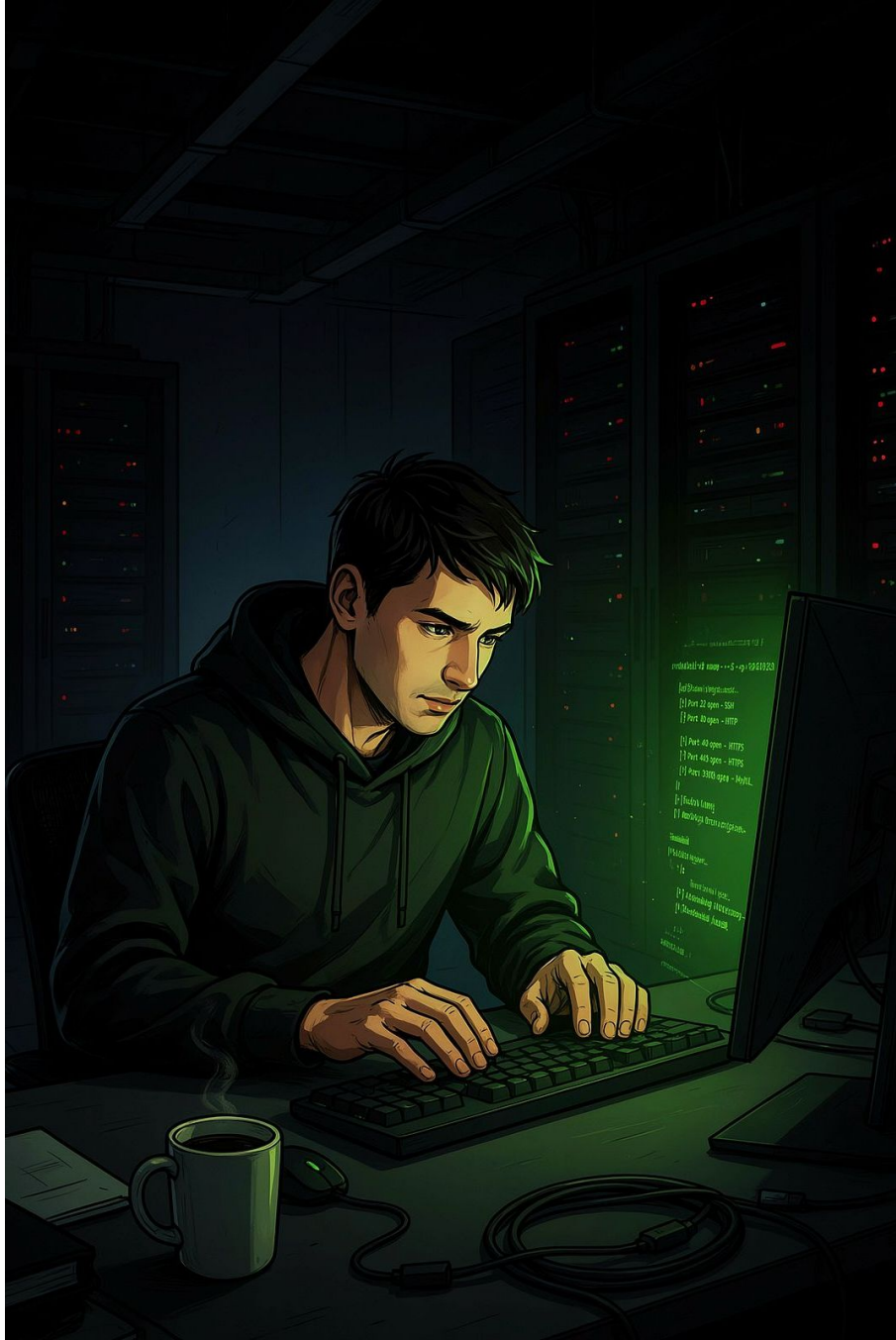


Registry

Хранилище образов. Публичный Docker Hub или приватный внутренний реестр организации.

РАЗДЕЛ 8

Установка Docker на Kali Linux



Установка Docker на Kali: общий порядок

1 Установить пакеты

Установите Docker из репозиториев Kali.

```
sudo apt update
sudo apt install -y docker.io docker-compose
```

2 Проверить установку

Запустите сервис и протестируйте тестовым контейнером.

```
sudo systemctl enable --now docker
sudo docker run hello-world
```

Проверка установки Docker

```
sudo systemctl enable --now docker
sudo docker run hello-world
docker compose version
```

Важно: Compose как плагин

Современный синтаксис — `docker compose` (через пробел). Устаревшая отдельная команда `docker-compose` не используется в новых установках.

Группа docker и безопасность

Добавление пользователя в группу `docker` позволяет работать без `sudo`, но фактически даёт привилегии уровня **root**. На учебной машине принимайте это решение осознанно.



РАЗДЕЛ 9

Сборка приложения: Dockerfile

Пример Dockerfile для Python API

```
FROM python:3.13-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN useradd --create-home appuser
USER appuser
EXPOSE 8000
CMD ["python", "app.py"]
```

Что здесь происходит

- `FROM` — базовый образ
- `WORKDIR` — рабочая директория
- `COPY` + `RUN` — установка зависимостей
- `useradd` / `USER` — запуск не от root
- `EXPOSE` — документация порта
- `CMD` — команда запуска

Сборка и запуск контейнера

```
docker build -t ctf-web-demo:1.0 .
docker run --rm -p 127.0.0.1:8000:8000 ctf-web-demo:1.0
```

> Привязка `127.0.0.1:8000:8000` делает сервис доступным **только на локальной машине**. Это безопаснее, чем публиковать порт на все сетевые интерфейсы (`0.0.0.0`), что открыло бы его в сеть.

Флаг `--rm` автоматически удаляет контейнер после остановки — удобно при разработке и тестировании, чтобы не накапливать остановленные контейнеры.



РАЗДЕЛ 10

Несколько сервисов с docker compose

Пример docker compose: приложение + PostgreSQL

```yaml
services:
  app:
    build: .
    ports:
      - "127.0.0.1:8000:8000"
    environment:
      DATABASE_URL: postgresql://ctfapp:change-me@db:5432/ctfdb
    depends_on:
      - db
  db:
    image: postgres:17-alpine
    environment:
      POSTGRES_DB: ctfdb
      POSTGRES_USER: ctfapp
      POSTGRES_PASSWORD: change-me
    volumes:
      - postgres_data:/var/lib/postgresql/data
volumes:
  postgres_data:
```

⚠ Пароль указан в файле только для наглядности. В реальном проекте используйте переменные окружения из защищённого хранилища или Docker Secrets.

Управление сервисами через compose

Запуск

```
docker compose up --build
```

Собирает образы и запускает все сервисы, описанные в файле.

Статус

```
docker compose ps
```

Показывает состояние каждого сервиса: запущен, остановлен, порты.

Логи

```
docker compose logs -f app
```

Выводит логи сервиса в реальном времени. Флаг `-f` — follow.

Остановка

```
docker compose down
```

Останавливает и удаляет контейнеры. Volumes сохраняются.